

Character Animation Lab

The goal of this lab is to explore standard character animation techniques to control a virtual character. In particular, our objective is to develop a MotionGraph, and to enable the character to follow a trajectory that you computed in the previous section of the course.

To this end, this lab is organised into two parts. In the first part, you will become familiar with similarity metrics, i.e., computing how similar two animations are. In a second part, you will be building a Motion Graph, as presented in the course.

Ressources

To help you in this lab, we are providing you with a number of ressources.

Animations. The folder *data/animations/* includes a number of animations downloaded from Mixamo that you will be using in the lab. Feel free to download more animations if you deem them necessary: <https://www.mixamo.com/#/?page=1&query=ybot&type=Character> (then select the “Find Animation” button to select and export animations for this character, or you can use another character if you prefer to do so).

Code. Most of the code that you will need to understand and complete is located in the folders *pynimation/anim/metrics* and *pynimation/anim/motiongraph*. Do not forget that the documentation is available from *docs/index.html*

Step 1 - Computing similarities between animations

The first step of this lab is to explore how to design similarity metrics. To explore these questions, you can start by looking at the *labs/lab_03_similarity.py* file.

To compute similarities between motions you can use the Similarity.py file:

```
sim = Similarity(YourMetric(), anim1=anim0,  
                skeletonMap=Skeleton.SkeletalPart.defaultSkeleton())
```

It uses custom metrics inheriting from the Metric.py file to define your own metric class.

```
class YourMetric(Metric):
    def prepareFrame(self,
        frameID: int,
        anim: "Animation",
        skeletonMap: Optional[List["Skeleton.SkeletalPart"]] = None
    ) -> List:
        #Add your own code here

    def evaluate(
        self, dataFrame1: List[Quaternion], dataFrame2: List[Quaternion]
    ) -> float:
        #Add your own code here
```

A custom metric should define the prepareFrame() function, which prepares the data to be evaluated for a given frame (e.g. precomputes all the global positions, or the global positions and their velocities, etc.) which returns a list dataFrame of elements to evaluate for this frame. It should also include an evaluate function, which computes the similarity metric given the dataFrame corresponding to two animation frames.

An example local_orientation_no_root.py is provided in the *pynimation/anim/metrics* folder for inspiration, which computes the metric based on the difference of local joint angles across joints.

To help you visualize the results, you can export and visualize similarity matrices as png images using

```
sim.toImage('./pynimation/data/similarity.png')
```

Action #1: create a novel custom metric which computes similarities based on the global position of the different joints. You will need to think to be invariant to the global position/orientation of the whole frame.

Action #2: create a second metric also taking joint velocities into account, and compare the difference with the previous metrics on different animations.

Bonus Actions: explore the creation of other metrics based on local/global positions, angles, and/or velocities.

Advices:

- When loading an animation, we recommend that you use the following function (**anim.skeleton.mapHumanoidSkeleton()**) to automatically detect the main joints of the skeleton. This will enable you to refer to specific joints as skeleton.root,

skeleton.head, ... (as defined in the skeleton.py file). This is already included in the *labs/lab_03_similarity.py* file.

- This will also enable you to use in the **YourMetric.evaluate(...)** function **skeletonMap=anim.skeleton.SkeletalPart.defaultSkeleton()**, in order to limit the comparisons between frames to a number of relevant joints (e.g., excluding fingers), which will speed up the computation process.
- The local/global transforms of a joint of a frame are accessible through *trans = frame.localTransforms[jointIndex]* and *trans = frame.globalTransforms[jointIndex]*.
- You can then access the different elements of the transform through dedicated functions (e.g., rotations using *trans.getQuaternion()* or *trans.getRotation()*, the position through *trans.getPosition()*, etc. - see documentation)

Step 2 - Build a motion graph

The second step of this lab is to explore how to design a Motion Graph enabling to make transitions between animations, in order to follow a specific trajectory. We ask you to demonstrate its usage on four motions which are provided, but you can also include more motions if you wish to.

Skeleton files are provided that you will need to fill in: *labs/lab_04_motiongraph.py* and *pynimation/anim/motiongraph*. You can refer to slide 30 in the *CG20XX_3_MotionControl.pdf* file for a summary of the main steps.

Action #1: Fill in the functions *_addMotion(anim)* and *_addMotionNodes(anim)*, to handle how to add a new motion to the motion graph

Action #2: Fill in the function *_addNewTransitions()*, to add the transitions between two specified motions. This requires to compute the similarity matrix between them, to compute local maxima, to threshold the values, etc.

Action #3: Prune the motion graph to keep the largest connected component (function *_prune* to fill)

Action #4: Test your motion graph with the random walk function, which will enable you to validate whether your motion graph is correct or not. While it is not usable to control a character, this is a good way of checking that your transitions create good visually looking animations.

Action #5: you can implement a graph traversal for path synthesis, so that your character now follows as closely as possible a provided trajectory, which can come from the previous part of the project.

Advice:

- To simplify the problem we are using already looping animations. These motions have been preprocessed so that the last frame is the same than the first frame, therefore a transition can automatically be added between the last frame and the second frame of the animation, which is a specific case you can handle by checking whether an animation is cyclic or not (`anim.isCyclic()`).
- After loading an animation, you will see that we use the function **`animation.setRelativeRootGlobalTransform()`** after loading an animation. It expresses the root transformation of a frame in the coordinate system of the previous frame, therefore describing the difference between the previous and the next frame global transformations. This is extremely handy when using a motion graph to describe the transformation from a frame to the next, and therefore to encode what a transition means in terms of transformation. By storing the cumulated root transform you will then be able to compute the current global root transformations as

```
myFrame = currentFrame.copy()
cumulatedTransform = numpy.dot(
    cumulatedTransform, myframe.localTransforms[myframe.skeleton.root.id]
)
myframe.localTransforms[fr.skeleton.root.id] = cumulatedTransform
```

Bonus Action #1: as poses at the time of transition might not be exactly the same, you can create a function that blends the difference between the poses over a series of frames (possibly adding new “blended” nodes in the motion graph).

Bonus Action #2: use additional animations downloaded from Mixamo to improve the capacities of your motion graph, which might be quite limited with the number and length of motions currently used (though be careful that these motions might not loop anymore, which you will therefore need to handle).